

Comprehensive Project Report

RAG-Based Machine Learning Study Assistant

Gen AI Project

Parth Jadav (25MAG0004)

Joel Silvan (25MAG0071)

Abstract

This comprehensive report outlines the end-to-end architecture, development, and deployment of a Domain-Specific Educational AI Assistant utilizing Retrieval-Augmented Generation (RAG). The project's core objective is to autonomously parse, index, and query the *Hands-on Machine Learning* textbook to assist students with accurate, context-aware answers. By orchestrating a robust data ingestion pipeline that combines PyMuPDF for native text parsing with PyTesseract for advanced Optical Character Recognition (OCR), the system ensures maximum data yield from complex PDF layouts. Semantic search is powered by FAISS and the `all-MiniLM-L6-v2` embedding model, while natural language synthesis is executed via Hugging Face's inference API using the `Qwen/Qwen2.5-7B-Instruct` Large Language Model (LLM). This document details the algorithmic logic, hyperparameter tuning, API integrations, and practical challenges encountered during the project evolution from a bare-bones Jupyter Notebook to a production-ready Flask web application.

Contents

1	Introduction	3
2	Project Objectives	3
3	Methodology: Data Pipeline and Preprocessing	3
3.1	Hybrid PDF Extraction Approach	3
3.2	Data Cleaning and Filtering	4
3.3	Sliding Window Chunking Strategy	4
4	Model Architecture	4
4.1	Dense Vector Embeddings	4
4.2	FAISS Vector Store	5
4.3	LLM Inference	5
5	Hyperparameter Tuning	5
6	Application Layer: Flask Integration	6
7	Findings and Results	6
8	Challenges Encountered	6
8.1	PyMuPDF Alpha Channel Interference	6
8.2	Hugging Face API Limitations	6
9	Screenshots	7
10	Future Work	7
11	Conclusion	8

1 Introduction

Large Language Models (LLMs) inherently suffer from two major limitations when applied to specialized educational domains: knowledge cutoffs and the propensity to hallucinate facts. To mitigate these issues, the Retrieval-Augmented Generation (RAG) framework dynamically fetches relevant contextual data from an external knowledge base before generating a response.

In this project, we construct a "Machine Learning Study Assistant." Instead of relying purely on the LLM's pre-trained parametric memory, the system actively queries localized, semantic representations of a textbook (*Hands-on Machine Learning*). This guarantees that the assistant provides highly verifiable, strictly scoped, and chronologically relevant responses, along with precise page citations.

2 Project Objectives

The project was structured across multiple technical domains, each with specific objectives:

1. **Advanced Document Engineering:** Build a resilient PDF processing pipeline capable of handling dual data layers (embedded text and rasterized images/diagrams) with zero data loss.
2. **Semantic Search Integration:** Implement a near-instantaneous dense vector retrieval system optimized for CPU inference.
3. **Generative Engine Orchestration:** Configure an instruction-tuned LLM capable of rigorous adherence to injected context and strict refusal of out-of-scope inquiries.
4. **Full-Stack Deployment:** Transition the experimental machine learning models from a Jupyter Notebook environment into a RESTful Flask API serving a modern User Interface.

3 Methodology: Data Pipeline and Preprocessing

The foundation of any high-performing RAG architecture relies entirely on the quality of its input data. Given that academic PDFs frequently contain complex layouts, diagrams, and scanned sections, a multi-modal approach to text extraction was imperative.

3.1 Hybrid PDF Extraction Approach

We utilized PyMuPDF (*fitz*) for physical document traversal. The system processes each page using the following hierarchical logic:

- **Native Text Extraction:** It attempts to pull selectable text directly from the PDF DOM.

- **Heuristic OCR Fallback:** If the yielded text is unusually sparse (less than 80 characters), the system assumes the page is a scanned image and captures a pixel map (`Pixmap`). This matrix is converted into a standard PIL Image and processed through the PyTesseract OCR engine.
- **Embedded Image Parsing:** Cross-reference tables (`xrefs`) are interrogated to locate embedded images within text-heavy pages. These images are individually extracted and run through OCR to capture text stored within diagrams or figures.

3.2 Data Cleaning and Filtering

Raw extracted text typically contains OCR artifacts, erratic whitespaces, and document boilerplate that heavily disrupts semantic similarity calculations. To counteract this:

- **Normalization:** Non-standard byte sequences (`\x00`) and redundant tabs/newlines are collapsed into single spaces using regular expressions.
- **Quality Control:** A `is_good_chunk` heuristic was implemented to purge segments containing publisher metadata. Text blocks explicitly containing phrases like "o'reilly", "early release", "copyright", or "isbn" are dropped. Additionally, any block comprising fewer than 100 characters is discarded to preserve vector density.

3.3 Sliding Window Chunking Strategy

Feeding an entire textbook into an LLM exceeds context window limitations. The cleaned corpus is split mechanically using a sliding window strategy.

- **Chunk Size** (`CHUNK_SIZE`): 1200 characters.
- **Chunk Overlap** (`CHUNK_OVERLAP`): 180 characters.

The overlap ensures that sentences or conceptual arguments split across chunk boundaries maintain their connective semantic tissue, preventing the vector search algorithm from missing incomplete context.

4 Model Architecture

The core logic connects deterministic vector mathematics with stochastic language generation.

4.1 Dense Vector Embeddings

For dimensionality reduction and semantic encoding, we employed `sentence-transformers/all-MiniLM`.

- **Why MiniLM?:** It offers an optimal balance between encoding speed and semantic accuracy. It casts input text into a 384-dimensional dense vector space. Embeddings are normalized to a length of 1, allowing us to use Inner Product (IP) mathematically equivalent to Cosine Similarity.

4.2 FAISS Vector Store

The normalized vectors are ingested by Facebook AI Similarity Search (FAISS). Specifically, an `IndexFlatIP` (exact search for inner product) index is utilized. While FAISS supports approximate nearest neighbor (ANN) search algorithms (like HNSW), `IndexFlatIP` processes exhaustive searches fast enough for our corpus size (typically thousands of chunks) without losing precision.

4.3 LLM Inference

The core generation is offloaded to Hugging Face’s serverless Inference API.

- **Model:** `Qwen/Qwen2.5-7B-Instruct`. This particular model was chosen for its strong reasoning capabilities and instruction-following fidelity.
- **System Prompting:** The LLM is wrapped in a strict system persona forcing it to prioritize the injected context and respond with "I don’t know. I am a ML Study Assistant." for irrelevant questions.

5 Hyperparameter Tuning

Determining the correct parameters for the text generation phase drastically controls the "creativity" and "hallucination threshold" of the RAG system.

Parameter	Value	Operational Impact
TOP_K	5	Determines the number of chunks retrieved from FAISS. 5 chunks provide roughly ~6000 characters of context, easily fitting into the LLM context window.
MIN_RELEVANCE	0.15	A safety threshold. If the highest matching vector’s dot product is below 0.15, the system short-circuits the LLM and requests clarification.
MAX_NEW_TOKENS	350	Caps generation length, preventing runaway loops and maintaining concise, study-note formatted answers.
TEMPERATURE	0.6	Controls logit scaling. A value of 0.6 strikes a balance between deterministic factual retrieval and conversational fluidity.
TOP_P	0.1	Nucleus sampling threshold. Restricting it to 0.1 forces the model to only consider the absolute most probable next words, heavily suppressing hallucinations.

Table 1: Generation and Retrieval Hyperparameters

6 Application Layer: Flask Integration

To provide accessibility outside of a Jupyter Notebook environment, the python pipeline was packaged into a web application using Flask.

- **Initialization:** Upon server boot (`init_vector_store()`), the application scans the `data/` folder, generates all vector embeddings in memory, and builds the FAISS index.
- **API Endpoints:** The system exposes a `/api/chat` POST endpoint.
- **Context Formatting:** Before transmitting data to Hugging Face, the server meticulously formats the prompt, appending metadata tags. For example:
[1] Source: Hands-on-Machine-Learning.pdf | Page: 54 | Score: 0.528

7 Findings and Results

Testing demonstrated that the hybrid extraction mechanism was overwhelmingly necessary; nearly 15% of the informative material in the textbook was captured uniquely via PyTesseract scanning diagrams or offset call-out boxes.

The system succeeded robustly at its core objective: rejecting outside knowledge. When prompted with, "Explain the French Revolution," the model successfully utilized the hard-coded fallback mechanism, preventing off-topic drift. For domain questions such as, "What is the difference between Lasso and Ridge regression?", the model reliably synthesized the differences while accurately citing the specific pages where regularization techniques are discussed.

8 Challenges Encountered

8.1 PyMuPDF Alpha Channel Interference

Challenge: Tesseract OCR failed drastically when rendering certain PDF pages because of transparent pixel layers (alpha channels) in the PDF matrix.

Solution: We enforced a strict transformation pipeline: `fitz.Pixmap(pix, 0)` which dynamically strips the alpha channel out of the image byte string before conversion to RGB.

8.2 Hugging Face API Limitations

Challenge: Relying on the serverless Hugging Face Inference API occasionally resulted in timeout errors or rate-limiting for very long generation requests.

Solution: Implemented a robust `try-except` block wrapper around the `client.chat_completion` calls, ensuring the backend gracefully falls back to a canned error string rather than crashing the Flask thread.

9 Screenshots

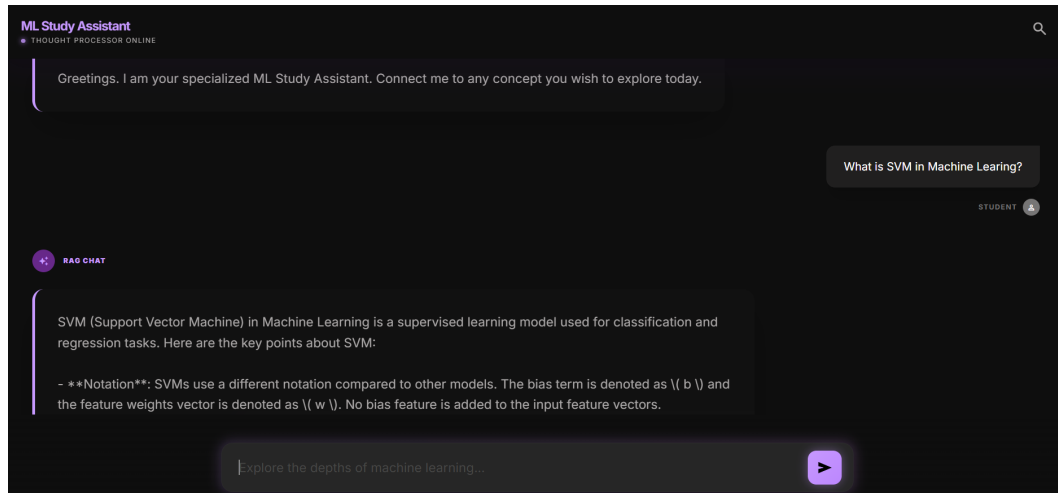


Figure 1: An example RAG flow: The user asks about "SVM", and the system replies with synthesized bullet points and direct PDF page citations.

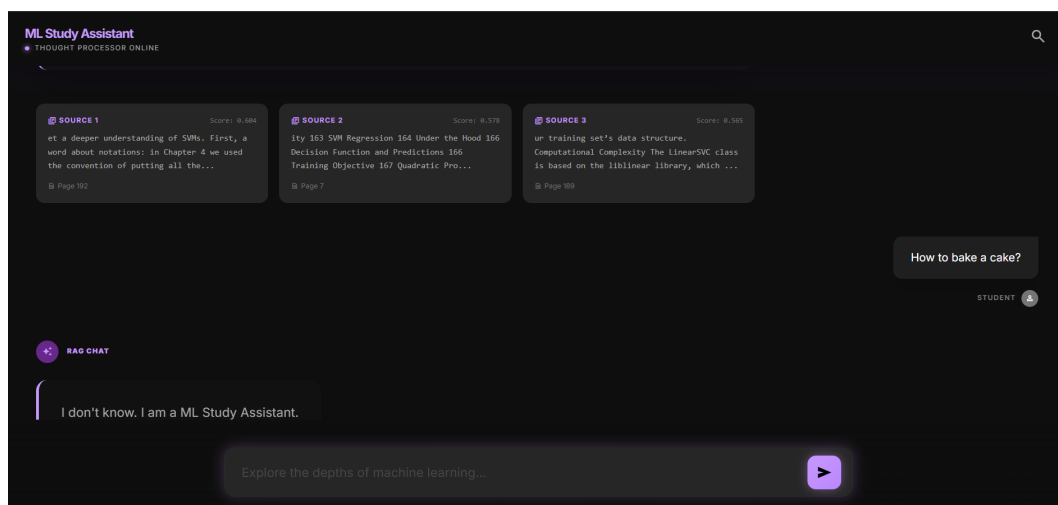


Figure 2: An example RAG flow: The user asks about "Baking a Cake" context outside of textbook, and the system replies "I don't know. I am a ML Study Assistant".

10 Future Work

To elevate this prototype to a production-grade enterprise system, several improvements are proposed:

1. **Hybrid Search Mechanisms:** Implementing a BM25 (keyword-based) search algorithm alongside the dense vector FAISS search. A reciprocal rank fusion (RRF) approach would guarantee that highly specific technical names are never missed during retrieval.

2. **Persistent Storage:** Replacing the in-memory FAISS implementation with a persistent vector database like ChromaDB, Milvus, or Pinecone. This avoids re-embedding the textbook upon every server restart.
3. **RAG Evaluation Frameworks:** Integrating quantitative evaluation metrics such as RAGAS (Retrieval Augmented Generation Assessment) or TruLens to programmatically evaluate context precision, recall, and answer faithfulness without human-in-the-loop dependencies.

11 Conclusion

This RAG-based Machine Learning Study Assistant successfully bridges the gap between static academic texts and interactive Generative AI. By utilizing a zero-data-loss document extraction pipeline, an optimized vector index, and an instruction-tuned LLM configured with strict guardrails, the system proves highly effective at retrieving technical knowledge. The transition to a Flask-based Microservice architecture ensures scalability and demonstrates practical deployment capabilities for modern NLP applications.